Factor-base designs: the two `m = 4` cells on a 256 GiB host (run-20261027T1318Z)

Kind: a measurement run, the rows of Amendment 14 (cells R256-A0 and R256-A11), run under Amendment 15 (checkpointed F4). Every number below is copied from `runs/analysis-amendment14.txt` (the output of `analyze --amendment14` on the committed rows), from the rows `runs/R256-A0-m4-t4.jsonl` and `runs/R256-A11-m4-t5.jsonl`, or from the logs beside them (`runs/R256-*.log`, `runs/R256-*.preflight.txt`); the few derived figures (percentages, ratios, differences) are marked as quotients. Nothing here prices a pipeline, reports S, or shows a speedup. A completed cell is a measurement of counted F_p multiplications in F4 row reduction; wall times and peak memory are descriptive. Split: not applicable (planted systems from C2, L1, H64: the identity check below shows these are the same systems). Leakage controls: the instrument and seeds are Amendment 14′s, unchanged; targets are planted by the cell seed, not drawn after seeing results.

Result

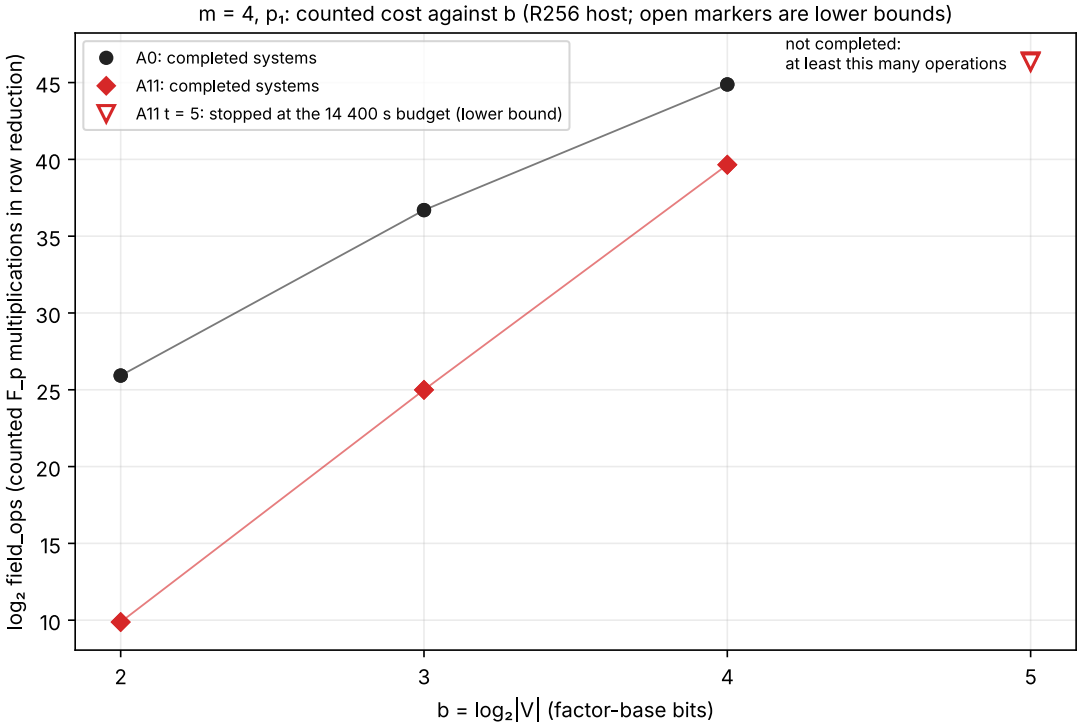


Figure 1: $\log_2$ counted field operations against $b = \log_2|V|$ at $m = 4$ on p_1. Filled markers: completed systems (A0 t = 2, 3 from C1, t = 4 from R256; A11 t = 2, 3, 4 from C1 and C2). Open triangles: the two R256-A11 systems at t = 5, stopped by the 14 400 s budget; their value is the operations spent by then, a lower bound. Source `runs/analysis-amendment14.txt`, `plot.py`.

cell	target	status	D	steps	max cols	field_ops	peak RSS (GB), time (s)
R256-A0 (t = 4)	0	ok	7	60	167 726	3.2354e13	121.2, 7 665 (F4 7 650)
R256-A0 (t = 4)	1	ok	7	60	167 726	3.2354e13	121.4, 7 727
R256-A11 (t = 5)	0	timeout	6 (reached)	20	477 822	9.2939e13 (so far)	223.0, 14 433
R256-A11 (t = 5)	1	timeout	6 (reached)	20	477 822	8.8755e13 (so far)	216.9, 14 434

Limit V = 259 227 4xx kB on every segment (MemTotal − 524 288 kB, no cgroup limit); the job cgroup recorded no OOM kill (oom_kill 0 before and after, no kernel OOM lines).

- **R1 (A11′s constant factor at m = 4, t = 4):** A11 t = 4 mean field_ops 8.630e11 ($\log_2$ 39.651) against A0 t = 4 mean 3.235e13 ($\log_2$ 44.879): **survives**, A11 is below A0 at t = 4. This is Amendment 14′s rule, copied from the analysis output.
- **A0 at m = 4 now has 3 completed cells:** $\kappa = \Delta\log_2$ field_ops$/\Delta b = 9.477$ against $m - 1 = 3$, $\sigma = 0.5000$: **cost-unfavourable**, no middle ground at this arity (the earlier two-cell value was 10.773, inconclusive).
- **R2 (A11 at t = 5 joins the fits): not measurable within 259 227 468 kB and 4 h**; the binding limit is time. The three-cell verdict stands: $\kappa = 14.886$, $\sigma = 1.5$, cost-unfavourable. Both systems had already spent 8.9–9.3 × 10^{13} counted operations when stopped, more than A0′s t = 4 systems needed in total (3.2 × 10^{13}); that is a count so far, not a value of the cell.
- **Identity:** all four R256 systems equal their counterparts in C2/L1/H64 (A0) and L2/L2r/H64 (A11) in cell seed, target seed, n_{vars}, n_{eqs}, planted x, curve and set.

Predictions (Amendment 14, fixed before any row)

- **R256-A0 "does not complete; binding limit memory near 6 100 s at about 251 GB": did not hold.** Both systems completed in about 7 700 s of F4 time at 121 GB, less than half of V. The peak-memory-in-time power law through two stops on different hosts, which was flagged as rough, did not extrapolate.
- **R256-A11 "does not complete; binding limit time, a timeout at 14 400 s, peak RSS near 90 GB": the first part held, the memory figure did not.** Both timed out at the F4 budget as predicted. Peak RSS was 223 GB and 217 GB, not about 90 GB: 86 % and 84 % of V (quotients of 223.0 and 216.9 GB by 259.2 GB). The analysis's own scorecard checks only completion and limit and prints "held" for this cell; the memory part of the prediction failed by a factor of about 2.5 (223.0 / 90).

How the run went (Amendment 15)

This is the reported record of what the checkpointing did and did not do; the rows above do not depend on any resume (see below).

- **Attempts before the rows.** Two builds failed on the host, a Spot reclaim destroyed about 73 min of R256-A0 before any checkpoint existed, and an unprotected instance was terminated after about 20 min. No row came from any of them. Amendment 15 added checkpoint and resume (F4Options::checkpointing), the self-test gate, and a persistent Spot request.
- **Self-test:** passed on the host before any cell: A0 m = 3 t = 4 plain against killed twice with SIGKILL and resumed, all counted fields equal (field_ops 15 920 614 779), 3 segments (`selftest.log`). Library test: `tests/f4_checkpoint_resume.rs`.
- **A0 target 0 is a resumed row:** 3 segments; the last resumed at step 52 from a checkpoint with 4 829 s of the budget already spent (32 189 basis elements, 242 399 pairs). Its field_ops equals what an uninterrupted run counts, by the test and the self-test above, not by comparison with a second full run of this system: no uninterrupted run of it exists. Its target 1 sibling (3.2354e13) completed in one uninterrupted segment and agrees with target 0 to 0.001 % (32 354 122 772 019 against 32 353 755 743 760), a different planted system, so this is consistency and not a replay.
- **A0 target 1 is not a resumed row:** its row comes from segment 4, which started from scratch (carried time 0): its three earlier segments were killed by Spot reclaims (at 01:18Z, and shortly before the boots at 10:58Z and 11:09Z) before any checkpoint was written, so none existed to resume from; the run restarted from the beginning each time. Checkpoints written in the final segment (40–49 GB) were never read.
- **Checkpoints are large and slow:** 40–49 GB for A0 and 55 and 101 GB for A11 (steps 18 and 19), written in 48–58 s (A0) and 62 and 101 s (A11) on a gp3 volume provisioned at 1000 MiB/s. At the default 125 MB/s the same write is about 6 min. The first checkpoint of a system appeared about 90 min after it started (A0 target 0: segment started 22:15Z, first checkpoint 23:47Z), so one early step must have lasted most of that, and a reclaim before then loses everything: four of the seven A0 segments were lost entirely this way (k0 segment 1; k1 segments 1, 2 and 3).
- **Run-script bug (erratum):** the first `run_amendment15.sh` skipped a cell as soon as it had any row. The host was reclaimed during A0 target 1 at 01:18Z, A0 was skipped on restart, and the target was run later with the fixed script (commit dfe40db2c; `--first-target 1`). A0 target 1′s 33 min lost in the first attempt are in no row.
- **V across segments:** each segment computed V from its own MemTotal; the six A0 values range over 259 227 456–259 227 476 kB (20 kB, 8×10^{-8} of V). Amendment 15 says segments that disagree on V make a cell not decided; this spread is a few kB of MemTotal rounding on the same instance type and no limit was reached, so no row depends on it, but it is a deviation from the letter of the rule and is reported as one.
- **OOM record:** `R256-A0-m4-t4.oom.txt` holds the last segment's record only (0 OOM events); earlier segments ended by host loss and wrote none. A0′s rows completed, so no limit needs attributing.
- **F4′s progress lines** reached the log only when a segment's process ended normally, so the checkpoint lines of killed segments are missing.

Source and cost

- **Binary:** built on the host from the tree of commit `ffcfcd8f1` (Amendment 14′s merged state) with the Amendment 15 files (`f4_fp.rs`, `fb_designs_slope.rs`, the two scripts) from commit `2d526ec61` of PR #2665. It is not a build of that commit's tree: main at the time did not compile (`binary_semaev_chain_sat.rs` had unbalanced braces after merge `4f1fa86ac`). sha256 `d750308a45a69334823ecb673bbafc98848dc8ce8647e81d3a64fba7709500c6`, rustc 1.99.1. The rows record `git_commit 2d526ec61`.
- **Host:** AWS EC2 r7i.8xlarge (Xeon Platinum 8488C, 32 vCPU, 256 GiB), us-west-2, Spot, Ubuntu 24.04; several instances of the same type under one persistent request (hostnames in the rows). The root volume was 100 GiB, grown to 800 GiB at 23:56Z (checkpoint plus its previous file plus a temporary file did not fit in 100 GiB).
- **Cost:** roughly 18 instance-hours at Spot prices of about $0.76–1.07 per hour, plus the larger volume (gp3 800 GiB, 16 000 IOPS, 1000 MiB/s for about 16 h): about $20–25, an estimate from those prices and not a bill.
- Data: rows and logs are in `runs/` the S3 copy is `s3://crypto-autoresearcher/lab-results/factor-base-designs-a14/2d526ec61-ckpt/state/out/` (rows, logs, pre-flight, OOM and self-test records; the 40–101 GB checkpoints were deleted after their systems finished).

What this does and does not show

- It shows: at $m = 4$ on p_1, A0′s $t = 4$ cell completes in 3.2×10^{13} counted operations and 121 GB, and its three-cell κ is 9.477 against the break-even 3; A11′s $t = 5$ cell does not complete in 4 h and 220 GB, and had already used more operations than A0′s whole $t = 4$ cell.
- It does not show: that A0 or A11 are cost-favourable ($\kappa \geq m - 1$ on every series with three cells, as before), anything at $m > 4$, larger t, or deployed sizes; nothing about timing on other hardware. No step of this is an improvement over any baseline; the rows are measurements of a negative result, and the ledger of improvements is unchanged.
- The A11 $t = 5$ operation counts are lower bounds, not values: R2 uses them only to say "not measurable", not to extend κ or σ.